

SCA TME 3&4

Delmotte Aurélien
Zhang Xiru

September 25, 2025

Contents

1	Introduction	2
2	Poids de Hamming et Modèle de Consommation	2
2.1	Identification de la phase SubBytes	2
2.2	Identification et analyse des points d'intérêt	3
3	Attaque par Analyse de Consommation Differentielle (DPA)	11
3.1	Réalisation de l'attaque DPA en considérant un bit	11
3.2	Analyse de l'efficacité de l'attaque de DPA single-bit	12
3.3	Attaque DPA Multi-bit	12
3.4	Analyse de l'efficacité des attaques de DPA multi-bit	13
4	Conclusion	13

1 Introduction

Les attaques par analyse de la consommation de puissance, en particulier les attaques différentielles par analyse de puissance (DPA), représentent une méthode efficace pour compromettre la sécurité des algorithmes de chiffrement symétrique tels que l’AES. Ces attaques exploitent les variations de consommation électrique liées au traitement des données secrètes pour inférer des clés cryptographiques.

Ce travail se concentre spécifiquement sur la phase de SubBytes de l’AES, une étape cruciale où chaque byte du bloc de données est substitué en utilisant une S-box non linéaire, ce qui est une source significative de variations de consommation d’énergie exploitables pour une DPA. L’identification précise des points d’intérêt (POI) dans cette phase est essentielle pour mener une attaque efficace.

Nous décrirons d’abord le processus d’identification des POI lors des opérations SubBytes en analysant les traces de consommation électrique. Ensuite, nous expliquerons comment ces POI sont exploités pour appliquer des attaques DPA, en visant à révéler non seulement des bits individuels de la clé mais également la clé complète. Notre méthode illustre la démarche à partir de la capture des traces jusqu’à l’analyse statistique détaillée, démontrant l’efficacité de l’attaque sur la phase SubBytes.

2 Poids de Hamming et Modèle de Consommation

2.1 Identification de la phase SubBytes

Le code AES a été modifié pour inclure des instructions nop avant et après la phase SubBytes pour clairement distinguer cette phase lors de la capture des traces d’exécution.

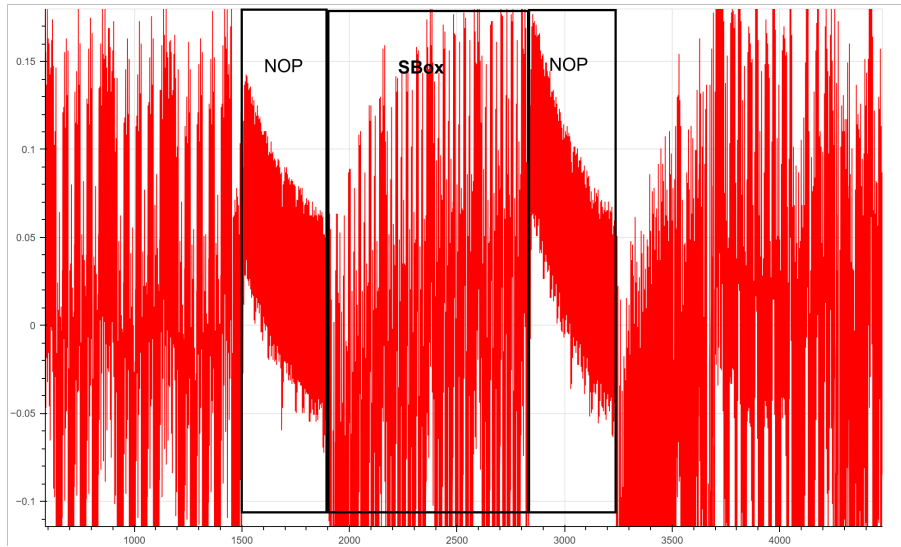


Figure 1: Trace de l’exécution avec les NOPs avant et après les opérations SubBytes.

La fonction `SubBytes` transforme chaque byte de l’état en utilisant une S-box. Le détail des cycles d’exécution pour chaque opération au sein de cette fonction est le suivant :

- **Transformation de chaque byte** : Chaque appel à la S-box pour transformer un byte nécessite 12 cycles, ce qui équivaut à 48 échantillons par byte.²
- **Entre les itérations de la boucle interne** : À la fin de chaque transformation de byte dans la boucle interne, et avant de passer au byte suivant, il y a un délai de 7 cycles, soit 28 échantillons, pour préparer la prochaine opération.³

- **Initialisation à l'entrée de la fonction** : Avant de commencer la transformation du premier byte, la fonction nécessite environ 12 cycles, soit 48 échantillons, pour l'initialisation et le positionnement sur le premier byte.⁴

Pour assurer la capture de l'intégralité de la phase SubBytes, il est nécessaire de calculer le nombre total d'échantillons requis. Le calcul est basé sur les besoins de chaque byte transformé et les délais entre eux :

$$\text{Échantillons requis} = 16 \times 48 + 3 \times 28 + 200$$

Ce total inclut un surplus de 200 échantillons pour garantir que les marges de début et de fin sont correctement couvertes, permettant ainsi une analyse complète sans perte d'information due à des coupures imprévues.

L'offset pour commencer la capture des traces est calculé pour aligner précisément avec le début de l'exécution de SubBytes. Étant donné que la phase commence environ à 1500 échantillons après le début de l'encryption, et que 48 échantillons supplémentaires sont nécessaires pour atteindre la première exécution effective de la S-box, l'offset ajusté est :

$$\text{Offset} = 1500 + 48 + 10 - 100$$

Les 100 échantillons retranchés permettent d'élargir la fenêtre de capture pour inclure d'éventuelles préparations ou initialisations précédant immédiatement la phase SubBytes.

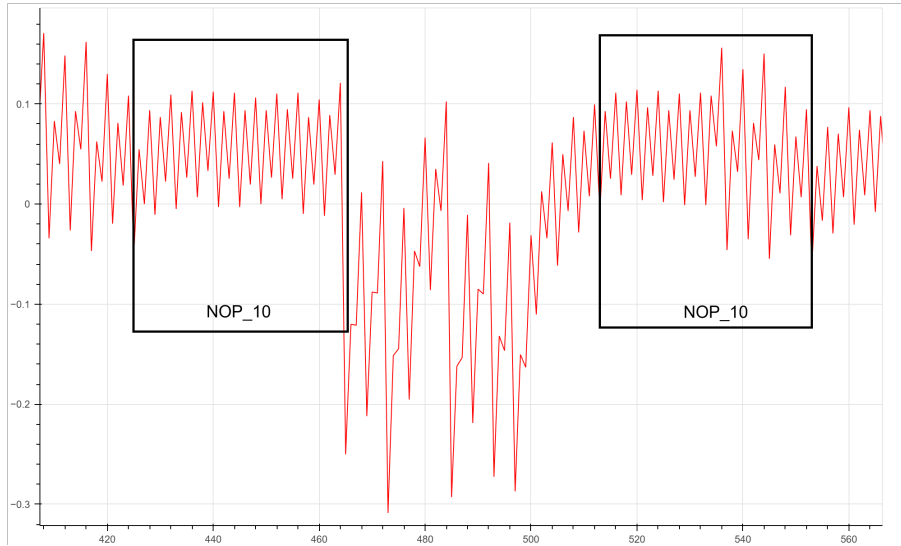


Figure 2: Graphique des échantillons nécessaires pour transformation de chaque byte (48 échantillons).

La capture des traces a été réalisée avec les paramètres offset à 1458 et samples à 1052. Ces paramètres ont été ajustés pour cibler précisément le traitement des 16 opérations SubBytes sans inclure d'autres opérations. Les figures 1 et 5 montrent respectivement les traces avec les nop et les traces se concentrant uniquement sur les calculs SBox.

2.2 Identification et analyse des points d'intérêt

Pour réaliser cette analyse, nous avons capturé un total de 2000 traces de consommation d'énergie lors de l'exécution de la phase SubBytes avec diverses entrées. Dans notre analyse, nous avons identifié des points d'intérêt (POI) où la consommation d'énergie est corrélée avec le poids de Hamming de la sortie de la S-box. Pour chaque byte de clé, du 0 au 15, nous avons examiné les traces pour repérer où la variation du poids de Hamming était la plus marquée et régulièrement gradée du rouge clair au rouge foncé.

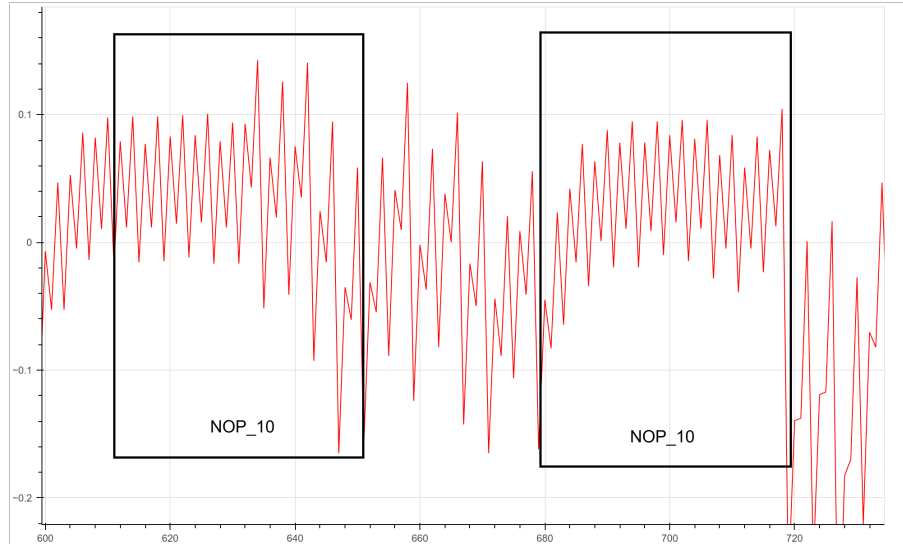


Figure 3: Entre les itérations de la boucle interne(28 échantillons).

Octet de clé 0	Octet de clé 4	Octet de clé 8	Octet de clé 12
148	196	244	320
Octet de clé 1	Octet de clé 5	Octet de clé 9	Octet de clé 13
368	416	464	540
Octet de clé 2	Octet de clé 6	Octet de clé 10	Octet de clé 14
588	636	684	760
Octet de clé 3	Octet de clé 7	Octet de clé 11	Octet de clé 15
808	856	904	936

Table 1: Tableau des positions des points d'intérêt pour chaque octet de clé

Les positions des POIs identifiés sont répertoriées dans le tableau ci-dessous:

Les graphiques suivants montrent les traces de consommation aux points d'intérêt identifiés, avec une gradation de rouge illustrant le poids de Hamming.

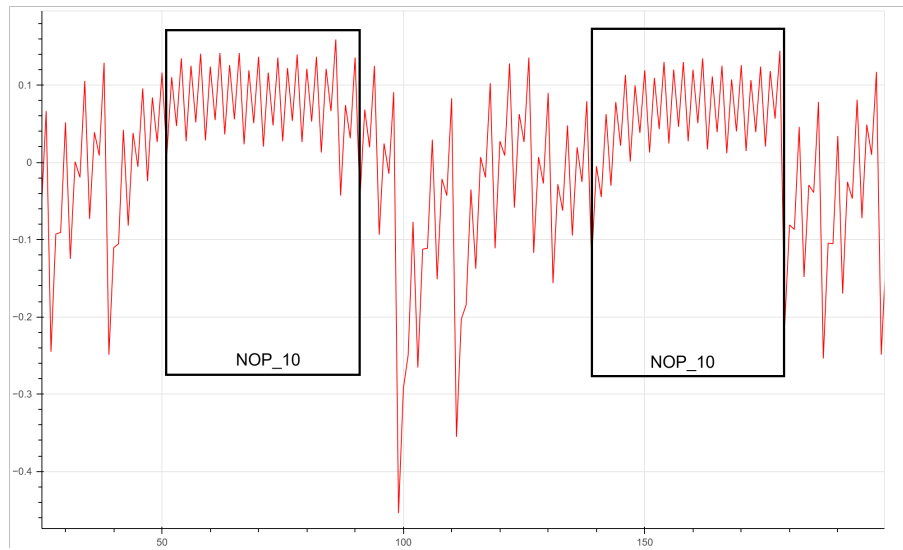


Figure 4: Initialisation à l'entrée de la fonction(48 échantillons).

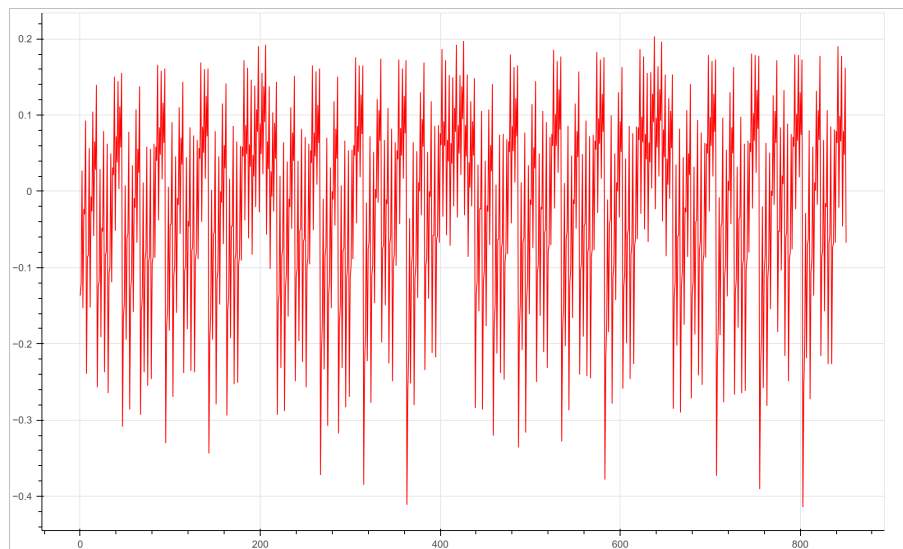
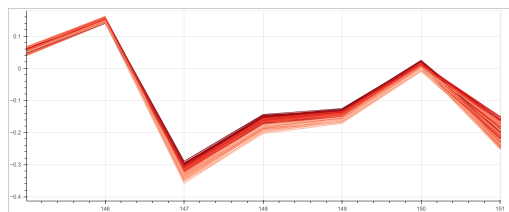
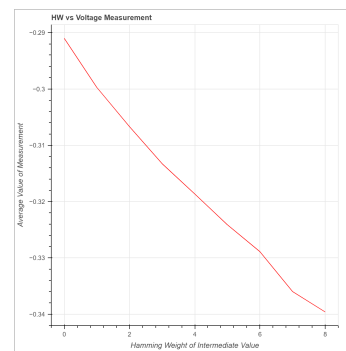


Figure 5: Trace de l'exécution focalisée uniquement sur les opérations SBox.

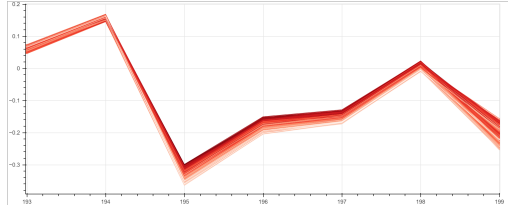


(a) Point d'intérêt à l'octet de clé 1

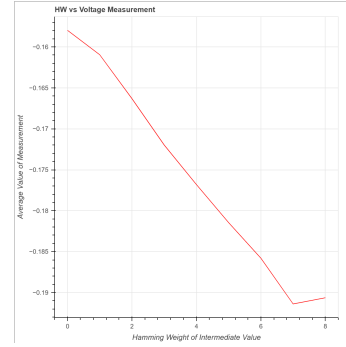


(b) Linearité à l'octet de clé 1

Figure 6: Octet de clé 1: Point d'intérêt à 148

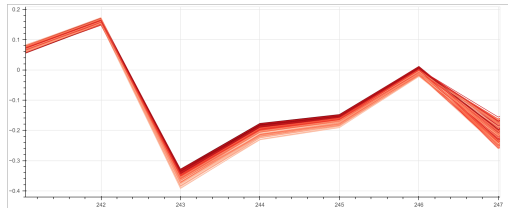


(a) Point d'intérêt à l'octet de clé 4

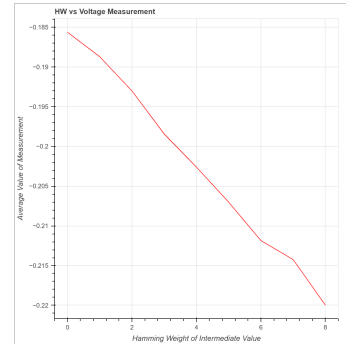


(b) Linearité à l'octet de clé 4

Figure 7: Octet de clé 4: Point d'intérêt à 196

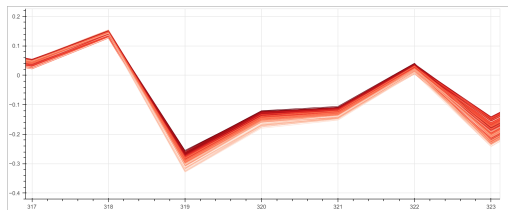


(a) Point d'intérêt à l'octet de clé 8

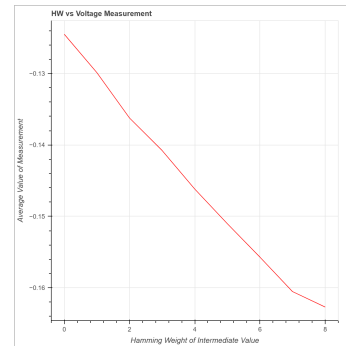


(b) Linearité à l'octet de clé 8

Figure 8: Octet de clé 8: Point d'intérêt à 244

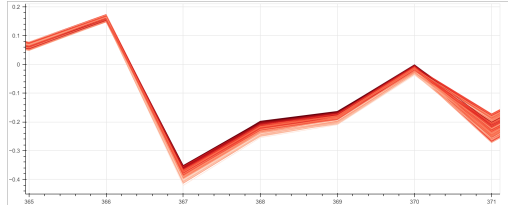


(a) Point d'intérêt à l'octet de clé 12

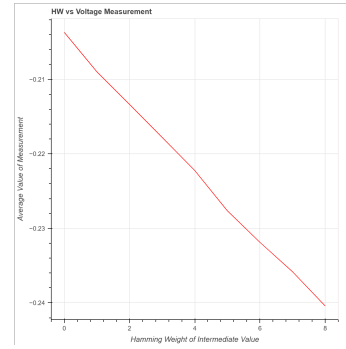


(b) Linearité à l'octet de clé 12

Figure 9: Octet de clé 12: Point d'intérêt à 320

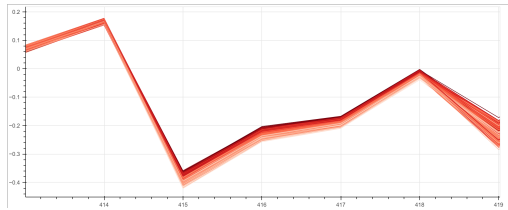


(a) Point d'intérêt à l'octet de clé 1

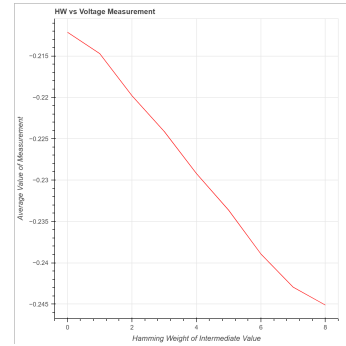


(b) Linearité à l'octet de clé 1

Figure 10: Octet de clé 1: Point d'intérêt à 368

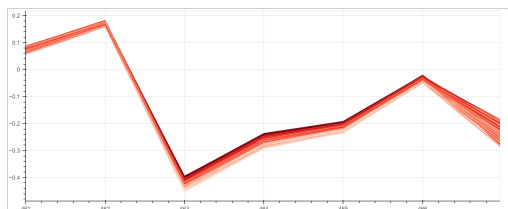


(a) Point d'intérêt à l'octet de clé 5

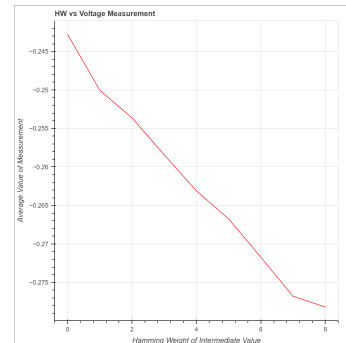


(b) Linearité à l'octet de clé 5

Figure 11: Octet de clé 5: Point d'intérêt à 416

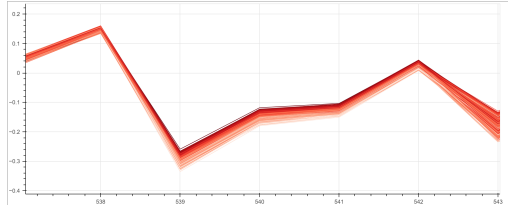


(a) Point d'intérêt à l'octet de clé 9

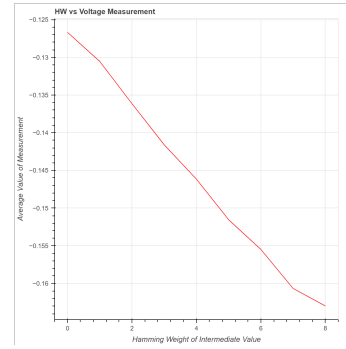


(b) Linearité à l'octet de clé 9

Figure 12: Octet de clé 9: Point d'intérêt à 464

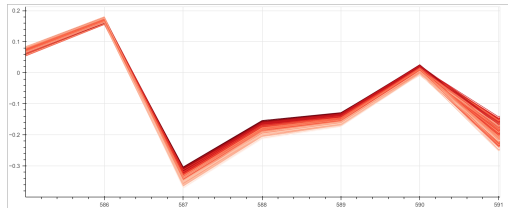


(a) Point d'intérêt à l'octet de clé 13

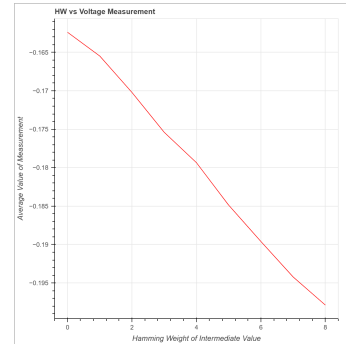


(b) Linearité à l'octet de clé 13

Figure 13: Octet de clé 13: Point d'intérêt à 540

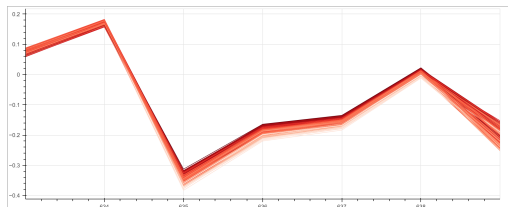


(a) Point d'intérêt à l'octet de clé 2

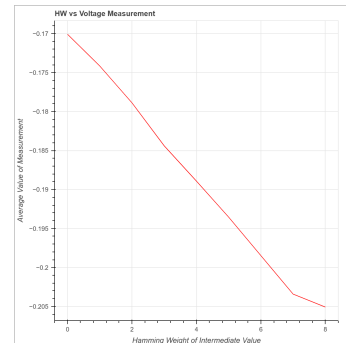


(b) Linearité à l'octet de clé 2

Figure 14: Octet de clé 2: Point d'intérêt à 588

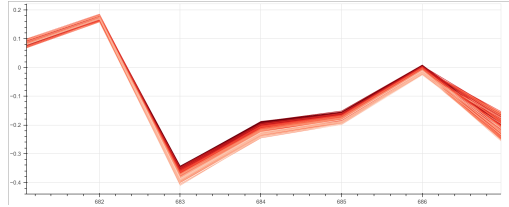


(a) Point d'intérêt à l'octet de clé 6

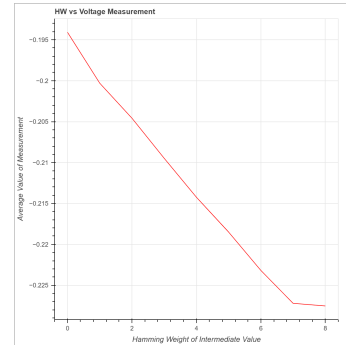


(b) Linearité à l'octet de clé 6

Figure 15: Octet de clé 6: Point d'intérêt à 636

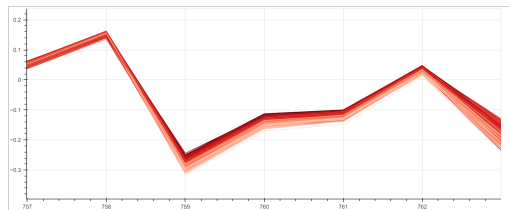


(a) Point d'intérêt à l'octet de clé 10

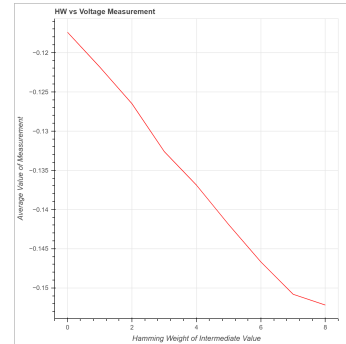


(b) Linearité à l'octet de clé 10

Figure 16: Octet de clé 10: Point d'intérêt à 684

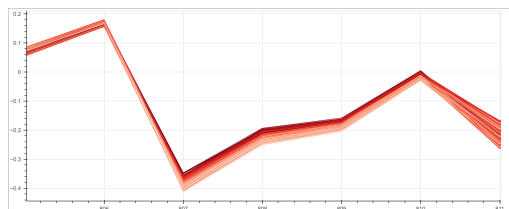


(a) Point d'intérêt à l'octet de clé 14

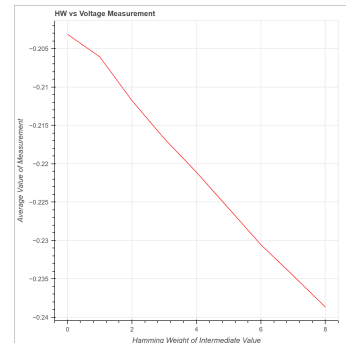


(b) Linearité à l'octet de clé 14

Figure 17: Octet de clé 14: Point d'intérêt à 760

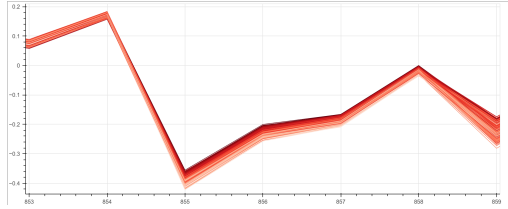


(a) Point d'intérêt à l'octet de clé 3

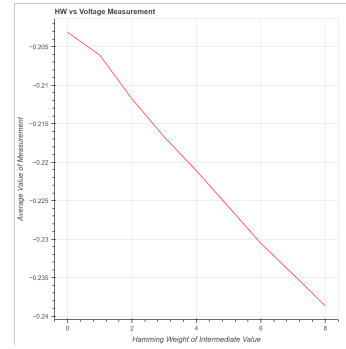


(b) Linearité à l'octet de clé 3

Figure 18: Octet de clé 3: Point d'intérêt à 808

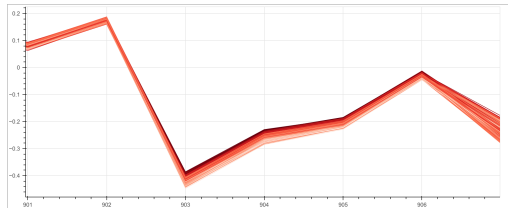


(a) Point d'intérêt à l'octet de clé 7

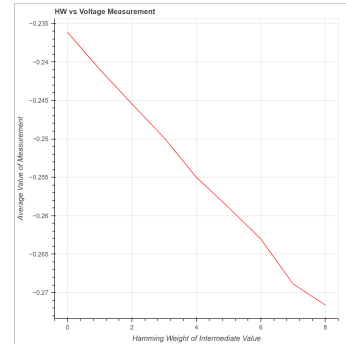


(b) Linearité à l'octet de clé 7

Figure 19: Octet de clé 7: Point d'intérêt à 856

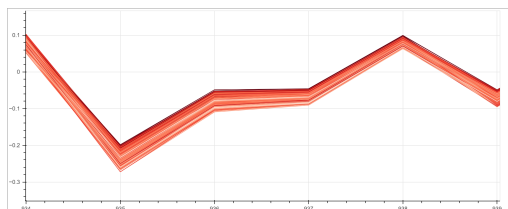


(a) Point d'intérêt à l'octet de clé 11

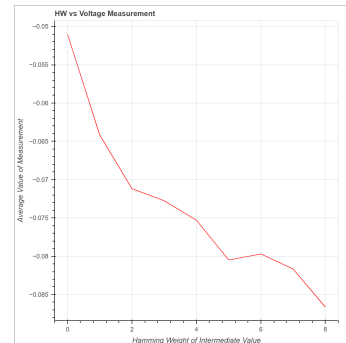


(b) Linearité à l'octet de clé 11

Figure 20: Octet de clé 11: Point d'intérêt à 904



(a) Point d'intérêt à l'octet de clé 15



(b) Linearité à l'octet de clé 15

Figure 21: Octet de clé 15: Point d'intérêt à 936

3 Attaque par Analyse de Consommation Differentielle (DPA)

L'attaque par analyse différentielle de la puissance (DPA) se base sur l'observation des variations de consommation de puissance lors de l'exécution de l'algorithme AES, en particulier pendant la phase SubBytes. Les étapes théoriques de notre attaque DPA sont les suivantes:

1. **Capture des traces de puissance** : Pour cette étude, nous avons collecté un total de 10,000 traces de consommation d'énergie pendant le chiffrement avec une clé fixe et des textes en clair variés. Ce grand nombre de traces vise à assurer une analyse robuste et fiable, permettant de capturer une variété suffisante de comportements cryptographiques pour l'analyse.
2. **Classification des traces** : Les traces sont classifiées selon la valeur d'un bit spécifique de la sortie de la SBox pour une sous-clé hypothétique. Les traces sont divisées en deux groupes, dépendant de la valeur de ce bit, pour évaluer la différence de consommation d'énergie induite par ce bit.
3. **Calcul de la différence des moyennes** : Nous calculons ensuite la différence entre les moyennes de consommation d'énergie des deux groupes de traces pour identifier la sous-clé la plus probable.
4. **Répétition pour chaque sous-clé possible** : Ce processus est répété pour chaque valeur possible de la sous-clé (0 à 255), afin de déterminer quelle sous-clé correspond à la plus grande différence des moyennes.
5. **Itération pour chaque byte de la clé** : L'opération est répétée pour chaque byte de la clé AES pour reconstruire progressivement la clé complète.

Pour évaluer et comparer l'efficacité des attaques DPA multi-bit et single-bit, nous avons utilisé un ensemble identique de 5000 traces pour chacune des méthodes.

3.1 Réalisation de l'attaque DPA en considérant un bit

L'attaque utilise la technique de différenciation basée sur le poids de Hamming des sorties des S-box. Pour chaque byte de la clé, nous effectuons une analyse statistique pour identifier la sous-clé qui maximise la différence des moyennes de consommation énergétique entre deux groupes de traces. Nous avons implémenté une fonction pour l'attaque DPA, qui vise à deviner chaque byte de la clé AES en analysant les différences de consommation d'énergie pour un bit spécifique de la sortie de la SBox. Le code suivant montre les fonctions utilisées pour cette attaque.

```
1 def guess_byte_single(traces,i,b):
2     j = i//4+(i%4)*4
3     poi = 936 if i==15 else (148+j*48 +(j//4+i//12)*28)
4     maxstd = 0
5     kg = 0
6     for k in range((2**8)):
7         hawk = [[],[]]
8         for trace in traces:
9             hawk[0 if intermediate(trace.textin[i],k)&b==0 else 1].append(trace.wave[poi])
10        std = abs(np.mean(hawk[0])-np.mean(hawk[1]))
11        if(std>maxstd):
12            maxstd = std
13            kg = k
14    return kg
15 def guess_key_single(traces,b):
16    key = bytearray(16)
17    for i in range(16):
18        key[i] = guess_byte_single(traces,i,b)
19    return key
```

Les résultats de l'attaque DPA sur un ensemble de 10,000 traces sont concluants. La clé correcte, comme indiquée dans les traces, est `2b7e151628aed2a6abf7158809cf4f3c`, et la clé devinée par notre algorithme était identique. Ce succès valide l'efficacité de notre méthode pour exploiter les variations de consommation d'énergie dues aux opérations SubBytes dans AES.

3.2 Analyse de l'efficacité de l'attaque de DPA single-bit

La Figure 22 ci-dessous illustre l'efficacité de l'attaque DPA Single-bit en fonction du nombre de traces utilisées pour chaque byte de la clé AES. Cette analyse montre clairement que la probabilité de succès augmente avec le nombre de traces. Cependant, il est également évident que certaines lignes, représentant certains bytes spécifiques, atteignent un taux de succès élevé plus rapidement que d'autres. Ceci peut être dû à des variations dans la sensibilité de ces bytes à l'attaque DPA ou à des différences dans la manière dont les informations sont traitées par l'algorithme AES.

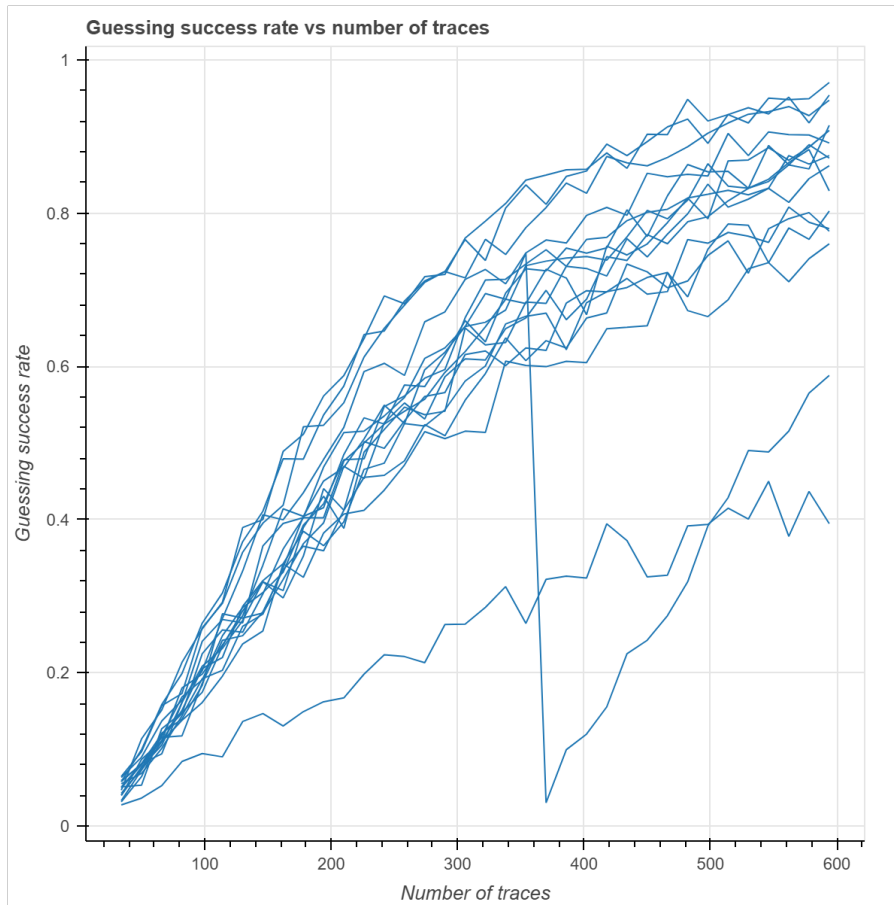


Figure 22: L'efficacité de l'attaque DPA Single-bit.

3.3 Attaque DPA Multi-bit

Contrairement à l'attaque DPA single-bit qui se concentre sur des bits individuels, l'attaque DPA multi-bit tire parti des informations combinées de plusieurs bits de la sortie de la S-box. Plus précisément, dans notre approche DPA multi-bit, nous classifions les traces de consommation selon le poids de Hamming du byte entier. La fonction `guess_byte_multi` exécute la DPA multi-bit en regroupant les traces en deux catégories basées sur le poids de Hamming des valeurs intermédiaires. Si le poids de Hamming de la sortie de la S-box est inférieur à 4, la trace est catégorisée dans un groupe; sinon, elle va dans l'autre groupe. Cette catégorisation repose sur l'hypothèse que différents groupes présenteront des modèles distincts de consommation d'énergie, influencés par le nombre de 1 dans le byte après la transformation de la S-box. La différence des moyennes

de consommation d'énergie entre ces groupes est ensuite calculée, et la supposition de clé qui maximise cette différence est considérée comme la plus probable.

```

1 def guess_byte_multi(traces,i):
2     j = i//4+(i%4)*4
3     poi = 936 if i==15 else 148+j*48 +(j//4+i//12)*28
4     maxstd = 0
5     kg = 0
6     for k in range((2**8)):
7         hawk = [[],[]]
8         for trace in traces[:200]:
9             hawk[0 if HW[intermediate(trace.textin[i],k)]<4 else
10                  1].append(trace.wave[poi])
11         std = abs(np.mean(hawk[0])-np.mean(hawk[1]))
12         if(std>maxstd):
13             maxstd = std
14             kg = k
15     return kg
16 def guess_key_multi(traces):
17     key = bytearray(16)
18     for i in range(16):
19         key[i] = guess_byte_single(traces,i,b)
20     return key

```

3.4 Analyse de l'efficacité des attaques de DPA multi-bit

La Figure 23 illustre l'évolution du taux de réussite des suppositions en fonction du nombre de traces utilisées dans notre attaque DPA multi-bit. Comme le montre le graphique, après une ascension initiale, la courbe atteint un plateau, ce qui suggère que le nombre de traces nécessaire pour obtenir un taux de réussite élevé ne doit pas nécessairement être extrêmement élevé. En effet, après environ 300 traces, le taux de réussite se stabilise, approchant une probabilité de succès de 100%. Cela démontre l'efficacité de l'approche multi-bit, qui, en utilisant une analyse combinée des plusieurs bits de chaque byte, permet de déchiffrer la clé avec une grande précision et moins de données comparativement à des méthodes nécessitant un plus grand nombre de traces.

En comparant ces résultats avec ceux obtenus lors de l'attaque DPA single-bit (voir Figure 22), on observe que l'approche Multi-bit tend à stabiliser le taux de succès à un niveau élevé avec moins de traces que l'approche Single-bit pour la majorité des bytes. Ce phénomène suggère que l'attaque Multi-bit, en exploitant des informations sur plusieurs bits simultanément, peut extraire des caractéristiques plus discriminantes des traces de consommation électrique, ce qui améliore l'efficacité globale de l'attaque.

4 Conclusion

Nous avons effectué une analyse détaillée pour identifier les 16 points d'intérêt (POI) dans la phase SubBytes de l'AES en utilisant des attaques par analyse de puissance différentielle (DPA). Les POIs ont été déterminés en fonction du poids de Hamming (HW) des valeurs intermédiaires calculées pendant le chiffrement, permettant une identification précise des instants critiques où la consommation énergétique révèle des informations clés.

Nos résultats montrent que l'approche multi-bit pour l'analyse DPA est supérieure à l'approche single-bit en termes d'efficacité et de rapidité pour identifier la clé cryptographique. En particulier, l'analyse multi-bit a démontré une capacité à stabiliser le taux de succès à un niveau élevé avec moins de traces que l'approche single-bit.

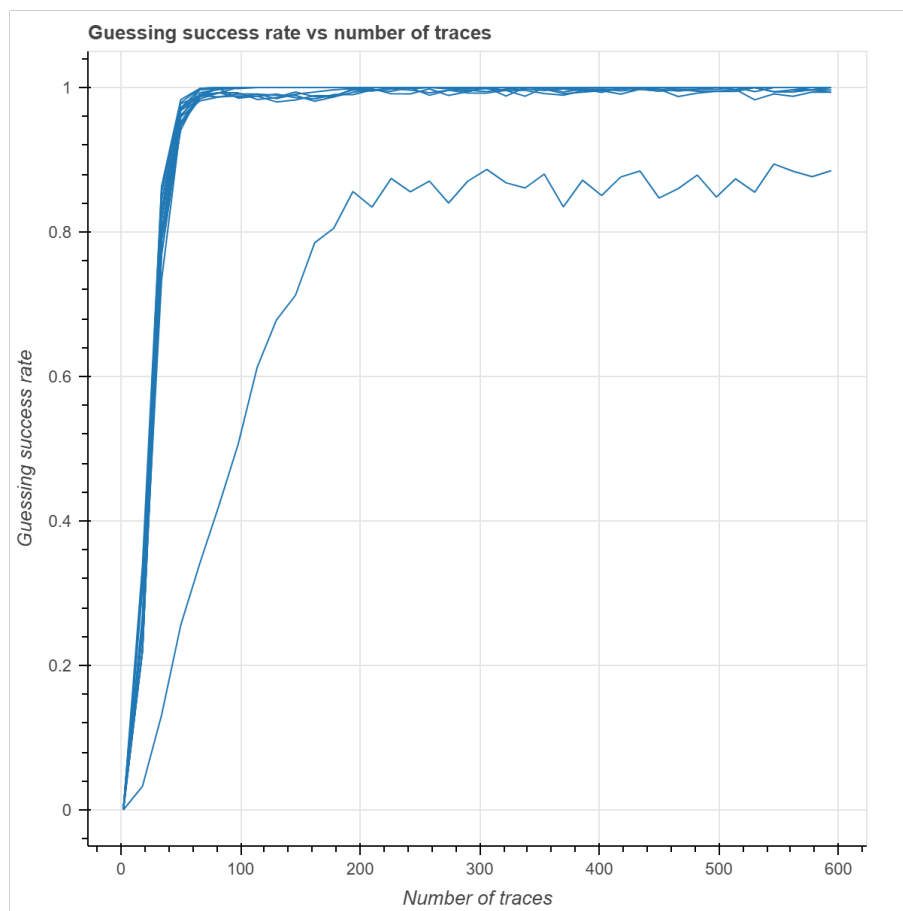


Figure 23: L'efficacité de l'attaque DPA multi-bit.